

IDEAS - Institute of Data Engineering, Analytics and Science Foundation

Summer Internship Program 2026

Project Title: Exploratory Data Analysis with Sales Data

Student Name: Dhruv Sharma

Course/Batch: Bachelor of Statistics (Honours/ Honours with Research), Batch: 2023–2027

Institute Name: Amity Institute of Applied Sciences, Amity University Uttar Pradesh, Noida

Period of Internship: 18th May 2026 - 14th June 2026

Report Submitted To: IDEAS - Institute of Data Engineering, Analytics and Science Foundation, ISI Kolkata

1. Abstract

This project presents an end-to-end framework evaluating operational retail telemetry alongside basic unsupervised digit representation clustering. Utilizing a comprehensive corporate sales framework consisting of historical columns (store_id, department, date, weekly_sales, is_holiday), we investigate core business parameters including high/low store revenue ranking, cross-department variance consistency, and the dynamic baseline shifts caused by national holiday profiles. Seasonal cycles were evaluated by parsing multi-year calendar variations to pin down high-performing months. Joint store-department operational nodes were mapped against corporate benchmarks to evaluate pareto concentration patterns. Concurrently, to satisfy technical curriculum structures, unsupervised partition testing was conducted using the standard MNIST pixel intensity matrix. A K-Means algorithm established structural separation across 10 native centers, evaluated with true digit mappings using macro-averaged F1 calculation criteria.

2. Introduction

In modern enterprise spaces, multi-node supply-chain operations generate immense operational logs that contain complex, underlying patterns. This industrial study bridges retail exploratory metrics and mathematical label sorting to extract clear business insights. The first section explores systemic market behaviors using standard tabular formats. It evaluates structural parameters like demand volatility, seasonal performance flags, holiday revenue changes, and revenue-concentration nodes across stores and departments. The second section focuses on raw mathematical representation mapping via unsupervised partition structures.

Training Program Record: During the initial two weeks of the internship, foundational instruction covered:

- Python Environment Operations & Object-Oriented Structures (Classes, Data Structures).
- Fast Matrix Math, Vector Operations, and Data Aggregation (Numpy & Pandas).
- Supervised Linear/Logistic Estimators & Error Backpropagation Fundamentals.
- Unsupervised Partitions, Centroid Calculations, and Objective Cluster Tracking (K-Means).
- High-Impact Technical Storytelling, avoiding jargon, and applying the Pyramid Principle.

3. Project Objective

- O1: Calculate corporate total revenues and categorize individual brick-and-mortar units into top and bottom performing tiers.
- O2: Measure seasonal variations and identify specific peak calendar windows by tracking month-by-month historical changes.
- O3: Standardize supply demand variability using statistical variance tests across all functional departments.

- O4: Quantify holiday sales shifts across different department categories to optimize seasonal inventory planning.
- O5: Benchmark extreme Pareto revenue-concentration coordinates across combined store-department operations.
- O6: Formulate unsupervised clustering using the standard MNIST dataset, mapping cluster labels back to their true values to calculate macro-averaged F1 evaluation scores.

4. Methodology

The analysis used an integrated data processing pipeline divided into two primary streams:

Stream A: Enterprise Sales Exploratory Pipeline

1. Ingestion & Integrity Check: Tabular retail data was loaded using the pandas framework. Missing records were verified and column datatypes were set correctly.
2. Aggregations & Groupings: Financial values were grouped by categorical indicators (store_id, department, month) using .groupby() statements to calculate sum, mean, and standard deviation.
3. Volatility Metrics: To control for scale bias across departments with different average sales volumes, the Coefficient of Variation (CV) was used as the primary consistency metric:

$$CV = \text{sigma}(\text{department}) / \text{mean}(\text{department})$$

4. Holiday Lift Analysis: Sales data was grouped by the is_holiday boolean flag to calculate the percentage change in average sales.

Stream B: Machine Learning Representation Pipeline

1. Dataset Loading: The standard MNIST digits framework was loaded as an 8*8 matrix (64 total feature elements).
2. Clustering Execution: A 10-center K-Means model partition was initialized and fitted directly on the pixel intensity matrix.
3. Label Re-mapping & Evaluation: Because unsupervised labels are assigned arbitrarily, a modal re-mapping loop matched each cluster to its most frequent true label. The final accuracy was measured using a macro-averaged F1 calculation:

$$\text{Macro F1} = \left(\frac{1}{N} \right) * (\text{sum of } F1_i \text{ from } i=1 \text{ to } N)$$

5. Data Analysis and Results

Section I: Descriptive Business Insights

- Store Revenue Rankings: Grouping total sales by store_id clearly separated high-performing locations from lower-revenue ones. Comparing average department sales within these top and bottom stores confirmed that high-performing stores sustain strong sales across all departments, rather than relying on a single top performer.

- Demand Stability Index: Sorting departments by their Coefficient of Variation highlighted highly stable categories (low CV, predictable inventory needs) versus highly volatile ones (high CV, prone to sudden demand shifts).
- Holiday Baseline Impacts: Analyzing the `is_holiday` flag showed a distinct increase in average weekly sales during holiday weeks. Certain consumer-focused departments showed massive sales lifts, making them primary targets for holiday promotional campaigns.
- Seasonal Demand Cycles: Monthly average sales plots highlighted distinct demand cycles throughout the year. Peak sales months point to high-demand shopping seasons, while low-sales months signal periods where inventory can be safely reduced.
- Pareto Concentration Nodes: Calculating percentage contribution for individual store-department pairs showed that a small number of combinations drive a disproportionate share of total corporate revenue.

Section II: Machine Learning Unsupervised Partitions

Running the K-Means clustering algorithm on the 1,797 image arrays in the MNIST dataset successfully partitioned the data across 10 distinct cluster labels.

Question 1: Import Data and Store-wise Revenue Analysis (4 Marks)

Import pandas and numpy. Download the dataset `sales.csv` from <https://drive.google.com/drive/folders/1gieHICVDBbUKMZiSF4YRQMAJic-w50JM?usp=sharing>. Load the dataset `sales.csv` into a DataFrame. Then, find the top 5 stores with the highest total sales and the bottom 5 stores with the lowest total sales.

Tasks:

- Aggregate total sales by store_id
- Rank stores by total revenue
- Compare average sales per department within those stores

```
# Import required libraries
import pandas as pd # to make data in table format
import numpy as np # to handle those data
import matplotlib.pyplot as plt # to plot those data & lines for visuilation

# Load the dataset sales.csv into a DataFrame
# (Assuming the file is downloaded locally or available in my working folder)
df = pd.read_csv("sales.csv")

# Task 1: Aggregate total sales by store_id
store_revenue = df.groupby("store_id")["weekly_sales"].sum().reset_index()

# Task 2: Rank stores by total revenue
store_revenue_sorted = store_revenue.sort_values(by="weekly_sales", ascending=False).reset_index(drop=True)

print("--- Top 5 Stores with Highest Total Sales ---")
top_5_stores = store_revenue_sorted.head(5)
print(top_5_stores)

print("\n--- Bottom 5 Stores with Lowest Total Sales ---")
bottom_5_stores = store_revenue_sorted.tail(5)
print(bottom_5_stores)

# Task 3: Compare average sales per department within those top and bottom stores
top_5_ids = top_5_stores["store_id"].tolist()
bottom_5_ids = bottom_5_stores["store_id"].tolist()

print("\n--- Top 5 Stores: Average Sales per Department ---")
top_dept_avg = df[df["store_id"].isin(top_5_ids)].groupby(["store_id", "department"])["weekly_sales"].mean().reset_index()
print(top_dept_avg.head(10)) # showing first 10 rows for display

print("\n--- Bottom 5 Stores: Average Sales per Department ---")
bottom_dept_avg = df[df["store_id"].isin(bottom_5_ids)].groupby(["store_id", "department"])["weekly_sales"].mean().reset_index()
print(bottom_dept_avg.head(10)) # showing first 10 rows for display
```

--- Top 5 Stores with Highest Total Sales ---

	store_id	weekly_sales
0	32	3.032230e+08
1	41	3.018119e+08
2	5	3.000639e+08
3	1	2.999050e+08
4	11	2.923540e+08

--- Bottom 5 Stores with Lowest Total Sales ---

	store_id	weekly_sales
45	13	60273453.26
46	34	46033475.43
47	12	38613465.75
48	8	31361162.23
49	2	27377684.86

--- Top 5 Stores: Average Sales per Department ---

	store_id	department	weekly_sales
0	1	1	62030.969103
1	1	2	65564.554423
2	1	3	69648.818654
3	1	4	76490.913910
4	1	5	76014.590321
5	1	6	81265.209551
6	1	7	81477.608397
...			
6	2	7	8141.880577
7	2	8	7791.310128
8	2	9	8033.990577
9	2	10	8673.889487

Question 2: Department Performance Consistency (3 Marks)

Which departments have the most stable weekly sales and which are the most volatile?

Tasks:

- Use standard deviation and coefficient of variation
- Identify highly stable and highly volatile departments

```
# Task 1: Use standard deviation and coefficient of variation to analyze consistency
# Group by department and calculate mean and standard deviation
dept_stats = df.groupby("department")["weekly_sales"].agg(["mean", "std"]).reset_index()

# Calculate Coefficient of Variation (CV = std / mean)
dept_stats["coefficient_of_variation"] = dept_stats["std"] / dept_stats["mean"]

# Task 2: Identify highly stable and highly volatile departments
# Sorting by CV: Low CV means stable, High CV means volatile
dept_stats_sorted = dept_stats.sort_values(by="coefficient_of_variation").reset_index(drop=True)

print("--- Top 5 Most Stable Departments (Lowest CV) ---")
print(dept_stats_sorted.head(5))

print("\n--- Top 5 Most Volatile Departments (Highest CV) ---")
print(dept_stats_sorted.tail(5))
```

```
--- Top 5 Most Stable Departments (Lowest CV) ---
  department      mean      std  coefficient_of_variation
0          13  60887.532814  46971.400142             0.771445
1           1   37200.611536  28781.821438             0.773692
2           2   39641.066051  30757.242670             0.775893
3           6   47729.393928  37045.408507             0.776155
4          14   63046.177503  48940.888590             0.776271

--- Top 5 Most Volatile Departments (Highest CV) ---
  department      mean      std  coefficient_of_variation
15          5  44963.078123  35413.433164             0.787611
16          9  53781.526778  42396.192677             0.788304
17         16  67908.462991  53568.802692             0.788838
18         20  75936.353251  60334.585409             0.794542
19         17  69205.177045  55319.077732             0.799349
```

Question 3: Holiday vs Non-Holiday Sales (3 Marks)

Analyze whether holidays significantly affect weekly sales.

Tasks:

- Compare average sales during holidays vs non-holidays
- Calculate percentage increase/decrease
- Identify departments benefiting most from holidays

```
# Task 1: Compare average sales during holidays vs non-holidays
holiday_comparison = df.groupby("is_holiday")["weekly_sales"].mean().reset_index()
print("--- Average Sales: Non-Holiday (0) vs Holiday (1) ---")
print(holiday_comparison)

# Task 2: Calculate percentage increase/decrease
non_holiday_avg = holiday_comparison.loc[holiday_comparison["is_holiday"] == 0, "weekly_sales"].values[0]
holiday_avg = holiday_comparison.loc[holiday_comparison["is_holiday"] == 1, "weekly_sales"].values[0]
pct_change = ((holiday_avg - non_holiday_avg) / non_holiday_avg) * 100
print(f"\nPercentage increase in sales during holidays: {pct_change:.2f}%")

# Task 3: Identify departments benefiting most from holidays
dept_holiday = df.groupby(["department", "is_holiday"])[["weekly_sales"].mean().unstack().reset_index()
# Rename columns for clarity
dept_holiday.columns = ["department", "Non_Holiday_Avg", "Holiday_Avg"]
# Calculate pure dollar increase for each department during holidays
dept_holiday["sales_lift"] = dept_holiday["Holiday_Avg"] - dept_holiday["Non_Holiday_Avg"]

dept_benefiting = dept_holiday.sort_values(by="sales_lift", ascending=False).reset_index(drop=True)
print("\n--- Top 5 Departments Benefiting Most From Holidays ---")
print(dept_benefiting.head(5))
```

```
--- Average Sales: Non-Holiday (0) vs Holiday (1) ---
   is_holiday  weekly_sales
0            0    52852.575328
1            1    84493.825439

Percentage increase in sales during holidays: 59.87%

--- Top 5 Departments Benefiting Most From Holidays ---
   department  Non_Holiday_Avg  Holiday_Avg  sales_lift
0            20      70978.560422  113946.098278  42967.537856
1            17      64309.266614  106740.490344  42431.223730
2            19      68665.759661  109712.451956  41046.692295
3            18      67139.112326  107698.553044  40559.440718
4            16      63435.025601  102204.816311  38769.790710
```

Question 4: Seasonal Trend Detection (3 Marks)

Identify whether sales increase or decrease during specific months.

Tasks:

- Extract month from the date column
- Compute average monthly sales
- Determine highest and lowest sales months

```
# Task 1: Extract month from the date column
df["date"] = pd.to_datetime(df["date"]) # ensuring standard datetime object format
df["month"] = df["date"].dt.month

# Task 2: Compute average monthly sales
monthly_sales = df.groupby("month")["weekly_sales"].mean().reset_index()

# Task 3: Determine highest and lowest sales months
monthly_sales_sorted = monthly_sales.sort_values(by="weekly_sales", ascending=False).reset_index(drop=True)

print("--- Monthly Performance Ranked ---")
print(monthly_sales_sorted)

highest_month = monthly_sales_sorted.iloc[0]
lowest_month = monthly_sales_sorted.iloc[-1]

print(f"\nHighest sales month is Month {int(highest_month['month'])} with average sales of {highest_month['weekly_sales']:.2f}")
print(f"Lowest sales month is Month {int(lowest_month['month'])} with average sales of {lowest_month['weekly_sales']:.2f}")
```

```
--- Monthly Performance Ranked ---
   month  weekly_sales
0      11  73301.879842
1      12  70916.599564
2       9  62113.261454
3       1  59409.675433
4      10  58702.346055
5       2  56327.288440
6       7  51878.715187
7       4  49509.001581
8       8  49289.191075
9       6  49130.168633
10      3  49096.696150
11      5  48642.593367

Highest sales month is Month 11 with average sales of 73301.88
Lowest sales month is Month 5 with average sales of 48642.59
```

Question 5: Store and Department Contribution (4 Marks)

Which combinations of store and department contribute the most to overall revenue?

Tasks:

- Group using store_id and department
- Find top and worst-performing combinations
- Compute percentage contribution

```
# Task 1: Group using store_id and department
comb_sales = df.groupby(["store_id", "department"])["weekly_sales"].sum().reset_index()

# Task 3: Compute percentage contribution to overall total corporate revenue
total_corporate_revenue = df["weekly_sales"].sum()
comb_sales["percentage_contribution"] = (comb_sales["weekly_sales"] / total_corporate_revenue) * 100

# Task 2: Find top and worst-performing combinations
comb_sorted = comb_sales.sort_values(by="weekly_sales", ascending=False).reset_index(drop=True)

print("--- Top 5 Store-Department Combinations by Total Revenue ---")
print(comb_sorted.head(5))

print("\n--- Worst 5 Store-Department Combinations by Total Revenue ---")
print(comb_sorted.tail(5))
```

✓ 0.0s

```
--- Top 5 Store-Department Combinations by Total Revenue ---
   store_id  department  weekly_sales  percentage_contribution
0         5           20  21949526.84             0.249015
1        32           19  21348232.29             0.242193
2         1           20  20751644.53             0.235425
3        41           20  20744989.41             0.235350
4        32           20  20494554.56             0.232508

--- Worst 5 Store-Department Combinations by Total Revenue ---
   store_id  department  weekly_sales  percentage_contribution
995         2           4   1060057.16             0.012026
996         8           1   1045511.61             0.011861
997         8           2   1037592.33             0.011771
998         2           2    929603.27             0.010546
999         2           1    903803.12             0.010254
```


Question 6: Load MNIST Data (3 Marks)

Load the MNIST dataset using `sklearn.datasets.load_digits`. Separate the dataset into features ($\bar{X}$) and target labels ($\bar{y}$). Print the shape of the features and the target arrays.

Expected Output: The shape of $\bar{X}$ and $\bar{y}$.

```
# Import MNIST loading mechanism from scikit-learn
from sklearn.datasets import load_digits

# Load the dataset
mnist = load_digits()

# Separate the dataset into features (X) and target labels (y)
X = mnist.data
y = mnist.target

# Print the shape of the features and target arrays
print(f"Shape of features array X: {X.shape}")
print(f"Shape of target array y: {y.shape}")
```

✓ 172s

Shape of features array X: (1797, 64)
Shape of target array y: (1797,)

Question 7: K-Means Clustering (5 Marks)

Import `KMeans` from `sklearn.cluster`. Initialize and fit the K-Means clustering model on the MNIST features ($\bar{X}$). Since we know there are 10 digits (0-9), set the number of clusters to 10. Assign the cluster labels to a variable `kmeans_labels`.

Expected Output: A successfully fitted K-Means model and the cluster labels array.

```
# Import KMeans estimator from scikit-learn library
from sklearn.cluster import KMeans

# Initialize K-Means clustering model with 10 clusters (digits 0-9)
kmeans = KMeans(n_clusters=10, random_state=42)

# Fit the model on the MNIST features (X)
kmeans.fit(X)

# Assign the cluster labels to a variable
kmeans_labels = kmeans.labels_

print("K-Means Model successfully fitted!")
print("Cluster labels array extracted:")
print(kmeans_labels[:50]) # Printing first 50 labels as an indicator sample
```

✓ 64s

K-Means Model successfully fitted!
Cluster labels array extracted:
[5 7 7 6 4 9 2 0 8 9 5 7 3 6 4 1 2 0 8 9 5 7 3 6 4 1 2 0 8 9 5 9 1 1 2 1 5
9 8 9 8 4 7 0 0 6 1 7 5 5]

Python

Question 8: F1 Score Evaluation for Clustering (5 Marks)

Evaluate the clustering performance using the F1 score. Since K-Means assigns arbitrary cluster labels, you will first need to map each cluster label to the most frequent true label in that cluster. After mapping the labels, calculate and print the macro-averaged F1 score using `sklearn.metrics.f1_score`.

Expected Output: The calculated F1 score of the clustering.

```
# Import target evaluation metrics from scikit-learn
from sklearn.metrics import f1_score
from scipy.stats import mode # to handle finding the most frequent labels

# Map each arbitrary cluster label to the most frequent true label in that cluster
mapped_labels = np.zeros_like(kmeans_labels)

for i in range(10):
    # Find positions where cluster equals i
    mask = (kmeans_labels == i)
    # Extract the true labels for these positions and find the mode
    if np.sum(mask) > 0:
        true_mode = mode(y[mask], keepdims=True).mode[0]
        mapped_labels[mask] = true_mode

# Calculate and print the macro-averaged F1 score
macro_f1 = f1_score(y, mapped_labels, average='macro')
print(f"Calculated Cluster-Mapped Macro-Averaged F1 Score: {macro_f1:.4f}")
```

✓ 0.0s

Calculated Cluster-Mapped Macro-Averaged F1 Score: 0.8628

Python

Mapping the cluster labels back to their true target counterparts and running the macro-averaged F1 test yielded a high-accuracy baseline, validating the model's ability to group similar handwritten digit shapes without pre-existing labels.

6. Conclusion

This exploratory and machine learning study yields two primary insights:

1. **Data-Driven Retail Operations:** Retail performance is highly concentrated, influenced by seasonal calendar cycles, and driven by distinct holiday demand shifts. Using these data-driven insights allows operations to optimize inventory levels based on department stability metrics and maximize high-contribution store-department nodes.
2. **Unsupervised Image Structuring:** The successful grouping of MNIST digit variations demonstrates the strength of centroid-based partitioning models. This confirms that the model can effectively capture structural variance and identify clear clusters even without manual training labels.

7. Appendices

1. **References:** Documentation for Pandas, Scikit-Learn Cluster & Evaluation Suite, and IDEAS Foundation Summer Training Manuals 2026.
2. **Code Repository Link:** https://github.com/dhruvsharma5603/14-Exploratory_data_analysis_with_sales_data_Summer_2026/blob/a95302a8af988dd09ae581a0639325e2a9d7aeb7/14_Exploratory_data_analysis_with_sales_data_Summer_2026_.ipynb